

Hamilog Project Documentation

A. Core Functions Tree

Frontend - React / Next.js / Tailwind

```
frontend/src/app
├─ layout.tsx
│   └─ RootLayout - Global website wrapper, ThemeProvider, and Navbar.
├─ page.tsx
│   └─ HomePage - Redirects to the login screen.
├─ login/page.tsx
│   ├── getSavedTheme - Reads the saved theme from the browser.
│   ├── applyTheme - Applies the theme to the html tag.
│   ├── LoginForm - Login and temporary driver registration form.
│   └─ LoginPage - Wraps the login form.
├─ dispatcher
│   ├── page.tsx
│   │   ├── getDisplayName - Returns the display name for a dispatcher user.
│   │   └─ DispatcherPage - Main dispatcher dashboard.
│   ├── menu/page.tsx
│   │   └─ DispatcherMenuPage - Dispatcher navigation menu.
│   └─ missions/page.tsx
│       ├── getInitialFiltersFromQuery - Reads filters from the URL.
│       ├── getMissionState - Calculates mission status for display.
│       ├── getStateClasses - Returns classes for status colors.
│       ├── getPriorityRank - Ranks priority for sorting.
│       ├── getIdealDeliveryIso - Builds an ISO target date.
│       ├── buildMissionPayload - Builds the payload for creating/updating a mission.
│       ├── toDateInputValue / toTimeInputValue - Convert dates into form fields.
│       ├── missionToForm - Converts an existing mission into an editing form.
│       └─ MissionsPage - Mission management page.
├─ alerts/page.tsx
│   ├── getLevelRank - Ranks alert severity.
│   └─ getRelatedAlertFilter - Determines if an alert is related to a mission/driver/
cargo/system.
│   ├── getCompatibleDrivers - Finds suitable drivers for a mission.
│   ├── buildAlerts - Builds the list of active alerts.
│   ├── groupAlertsForPopup - Merges alerts into popups.
│   └─ AlertsPage - Alerts page including filtering and dismissal.
├─ schedule/page.tsx
│   ├── getDeliveryState - Calculates mission status for the schedule.
│   ├── toLocalDateInputValue - Converts a date into a local input value.
│   ├── formatReadableDate - Displays a readable date.
│   └─ isSameLocalDate - Checks if a mission falls on the selected date.
```

- | | └─ TodaysSchedulePage - Dispatcher schedule and calendar page.
- | └─ reports/page.tsx
 - | | └─ getInitialDate - Returns a start date based on a given number of days back.
 - | | └─ DispatcherReportsPage - Dispatcher reports and charts.
- | └─ live-map/page.tsx
 - | | └─ getDriverForMission - Returns the driver assigned to a mission.
 - | | └─ LiveDeliveryMapPage - Live delivery map.
- | └─ drivers/page.tsx
 - | | └─ getInitialStatusFilter - Reads status filters from the URL.
 - | | └─ DriversPage - Driver management and filtering.
- | └─ drivers/[driverId]/history/page.tsx
 - | | └─ getParamString - Normalizes URL parameters.
 - | | └─ getDeliveryState - Calculates mission status within driver history.
 - | | └─ DriverHistoryPage - Driver history view for the dispatcher.
- | └─ pending-requests/page.tsx
 - | | └─ toMatchPercentage - Converts score to a match percentage.
 - | | └─ groupRequestsByMission - Groups requests by mission.
 - | | └─ PendingRequestsPage - Driver requests for missions.
- | └─ settings/help/messages/cargo-analysis - Settings, help, messages, and cargo analysis pages.
- | └─ driver
 - | | └─ page.tsx
 - | | | └─ DriverDashboardPage - Driver dashboard.
 - | | └─ availability/page.tsx
 - | | | └─ DriverAvailabilityPage - Driver availability calendar for marking/removing dates.
 - | | └─ menu/page.tsx
 - | | | └─ DriverMenuPage - Driver menu.
 - | | └─ history/page.tsx
 - | | | └─ DriverHistoryRoute - Redirects to driver history.
 - | | └─ messages/my-missions/open-tasks/profile/settings/help - Additional driver pages.

```
frontend/src/components
```

- └─ messages
 - └─ MessagesPage - Shared inbox page for both driver and dispatcher.
 - └─ MessageFilters - Tabs and filters for people.
 - └─ MessageDirectory - Contact list.
 - └─ MessageParticipantEntry - Contact card featuring a message button.
 - └─ MessageConversationPage - Conversation page with a specific user.
 - └─ MessageThread - Renders chat bubbles and handles message sending.
- └─ dispatcher
 - └─ alerts/AlertEntry - Expanded alert card.
 - └─ alerts/AlertPopup - Alert popup.
 - └─ dashboard/AlertsPanel - Dashboard alerts.
 - └─ dashboard/DriverStatusPanel - Driver status.
 - └─ dashboard/PendingRequests - Pending requests.
 - └─ live-map/LeafletDeliveryMap - Leaflet map component.
 - └─ missions/MissionEntry - Mission card.
 - └─ reports/* - Report and chart components.
 - └─ schedule/ScheduleEntry - Mission row inside the schedule.

- └ driver
 - └ DriverMissionWorkspace - Driver mission workspace environment.
 - └ missions/ActiveMissionCard - Active mission.
 - └ missions/OpenTaskCard - Open task.
 - └ tabs/MyMissionsTab / OpenTasksTab / ProfileTab - Driver page tabs.
 - └ history/* - Driver history components.
- └ shared
 - └ Calendar - Shared calendar for both driver and dispatcher.
 - └ SettingsPanel - Theme, font, sound, and notification settings.
 - └ SupportTicketPanel - Creation of support tickets.
 - └ ChatBot - Help chatbot.
 - └ BackToMenuButton - Button to return to the menu.
 - └ ThemeToggleButton / TextToSpeechButton / MenuCard / HelpPage.

```
frontend/src/lib
└ api-client.ts
  └ login - Authentication and JWT acquisition.
  └ getMissions / createMission / updateMission / updateMissionStatus / cancelMission.
  └ getDrivers / getDriver / updateDriverAvailability.
  └ getMissionRequests / createMissionRequest / approveMissionRequest /
  declineMissionRequest.
  └ getMessageParticipants / getMessageConversations / getMessages / sendMessage /
  markMessagesRead.
  └ createDriverRequest / approveDriverRequest / declineDriverRequest.
  └ analyzeCargo / sendChatbotMessage / createSupportTicket.
  └ getToken / setToken / clearToken / getStoredUser / setStoredUser.
└ driver-metrics.ts - Score, deliveries, and timeline calculations.
└ mission-distance.ts - KM calculation for missions.
└ mission-time.ts - Target and completion dates for missions.
└ mission-alerts.ts - Detects driver's last-minute cancellation.
└ sounds.ts - Plays sounds and mutes notifications.
└ date-format.ts - Date and time formatting utilities.
```

Backend - FastAPI / MongoDB

```
backend/app
└ main.py
  └ create_app - Creates FastAPI instance, configures CORS, and links routers.
└ core/security.py
  └ hash_password - Password encryption.
  └ verify_password - Password verification.
  └ get_seed_users - Generates development seed users.
  └ create_jwt - JWT token generation.
  └ decode_jwt - Decodes and verifies JWT tokens.
  └ get_current_user - Extracts the user from the token.
  └ require_role - Role-based permission checks.
└ database/state.py
  └ Chooses MongoDB, and falls back to InMemoryDB if no connection is available.
```

- └ database/mongo_store.py
 - | └ MongoDB.__init__ - Connection to Mongo, collections, indexes, and seeding.
 - | └ get/create/update missions.
 - | └ get/create/update drivers.
 - | └ create/list messages.
 - | └ mark_conversation_messages_read.
 - | └ create/list support tickets.
 - | └ driver/mission requests CRUD.
- └ database/memory_store.py
 - | └ Temporary development DB when MongoDB is unavailable.
- └ features/auth/router.py
 - | └ login - User login authentication.
- └ features/missions/router.py
 - | └ list_missions / create_mission / get_mission.
 - | └ update_mission_status.
 - | └ cancel_mission.
 - | └ update_mission.
- └ features/drivers/router.py
 - | └ list_drivers / get_driver.
 - | └ update_driver_availability.
 - | └ create_driver_request.
 - | └ approve_driver_request / decline_driver_request.
 - | └ _review_driver_request.
- └ features/drivers/service.py
 - | └ calculate_delivery_distance_km.
 - | └ mission_delivered_score.
 - | └ mission_cancelled_score.
 - | └ calculate_driver_trust_score.
 - | └ calculate_driver_score_for_mission.
- └ features/assignments
 - | └ check_driver_mission_compatibility.
 - | └ get_compatible_missions.
 - | └ calculate_match_score.
- └ features/messages/router.py
 - | └ create_message.
 - | └ list_conversations.
 - | └ list_message_participants.
 - | └ list_messages.
 - | └ mark_conversation_read.
- └ features/mission_requests/router.py
 - | └ create_mission_request.
 - | └ list_mission_requests.
 - | └ approve_mission_request.
 - | └ decline_mission_request.
- └ features/cargo/router.py
 - | └ analyze_cargo - Cargo text analysis.
- └ features/support_ticket/router.py
 - | └ create_support_ticket.
- └ routers/chatbot.py
 - | └ chatbot - Chatbot responses.

```

└─ realtime
  └─ websocket_manager.py - WebSocket connections management.
  └─ router.py - ws/gps and ws/dispatch endpoints.

```

B. API Details

Server Internal API

Method	Endpoint	Description / Role
POST	/auth/login	Login and receive JWT
GET	/health	Server health check
GET	/api/missions	Fetch missions
POST	/api/missions	Create a mission
GET	/api/missions/{mission_id}	Fetch a single mission
PUT	/api/missions/{mission_id}	Update mission details
PUT	/api/mission/{mission_id}/status	Change mission status
POST	/api/missions/{mission_id}/cancel	Cancel a mission
POST	/api/assign	Assign a mission to a driver
GET	/api/drivers	Fetch drivers
GET	/api/drivers/{driver_id}	Fetch a driver
PUT	/api/drivers/{driver_id}/availability	Update driver availability
GET	/api/driver-requests/pending/count	Number of pending driver requests
GET	/api/driver-requests	Fetch driver registration requests
POST	/api/driver-requests	Create a driver registration request

Method	Endpoint	Description / Role
POST	/api/driver-requests/{request_id}/approve	Approve driver request
POST	/api/driver-requests/{request_id}/decline	Decline driver request
POST	/api/mission-requests	Driver requests to accept a mission
GET	/api/mission-requests	Fetch mission requests
POST	/api/mission-requests/{request_id}/approve	Approve mission request
POST	/api/mission-requests/{request_id}/decline	Decline mission request
POST	/api/analyze-cargo	Analyze cargo text
POST	/api/messages	Send a message
GET	/api/messages/conversations	Get conversation list
GET	/api/messages/participants	Get message participant list
GET	/api/messages/{participant_role}/{participant_id}	Messages exchanged with a participant
POST	/api/messages/{participant_role}/{participant_id}/read	Mark messages as read
POST	/api/support-ticket	Create a support ticket
POST	/api/chatbot	Send a message to the chatbot
WS	/ws/gps/{driver_id}	WebSocket for driver GPS updates
WS	/ws/dispatch	WebSocket for dispatcher updates

External APIs and Services

Service	Usage	File Location
MongoDB / MongoDB Atlas	Stores users, drivers, missions, messages, requests, tickets	backend/app/database/mongo_store.py
OpenAI API	Cargo analysis / possible key-based fallback	backend/app/features/cargo/service.py, backend/requirements.txt
Google GenAI	Chatbot powered by GOOGLE_API_KEY (if defined)	backend/app/routers/chatbot.py, backend/requirements.txt
Leaflet / React Leaflet	Live map display	frontend/package.json, frontend/src/components/dispatcher/live-map
jsPDF + jspdf-autotable	Exporting reports to PDF	frontend/package.json, frontend/src/components/dispatcher/reports/shared/Utils.ts
Browser localStorage/sessionStorage	Stores token, user, theme, preferences, seen alert popups	frontend/src/lib/api-client.ts, frontend/src/lib/sounds.ts, frontend/src/lib/alert-popup-storage.ts

C. DB Connection

The MongoDB connection is configured via environment variables:

```
MONGO_URI=<mongodb connection string>  
DB_NAME=hamilog
```

```
JWT_SECRET=<secret>
OPENAI_API_KEY=<optional>
CORS_ORIGINS=http://localhost:3000
```

Example configuration file:

```
backend/.env.example
```

Connection implementation code:

```
mongo_uri = os.getenv("MONGO_URI")
db_name = os.getenv("DB_NAME", "hamilog")
self.client = MongoClient(mongo_uri, serverSelectionTimeoutMS=8000)
self.database = self.client[db_name]
```

Main Collections:

```
missions
drivers
driver_requests
mission_requests
messages
support_tickets
users
```

If a valid MongoDB connection is unavailable, the system automatically falls back to InMemoryDB for development purposes:

```
try:
    db = MongoDB()
except Exception:
    db = InMemoryDB()
```

| D. Special Environments / Installation

Frontend

```
Node.js
Next.js 16.2.9
React 19.2.4
Tailwind CSS 4
Leaflet / React Leaflet
```



```
jsPDF / jspdf-autotable
TypeScript
ESLint
```

Commands:

```
cd frontend
npm install
npm run dev
```

Frontend Environment Variables:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_ALLOWED_DEV_ORIGINS=192.168.1.119
```

NEXT_ALLOWED_DEV_ORIGINS is meant for development via a mobile phone on the same Wi-Fi network and is saved in `frontend/.env.local` (which is excluded from Git).

Backend

```
Python 3.12
FastAPI
Uvicorn
Pydantic
PyMongo / Motor
PyJWT
Passlib
python-dotenv
OpenAI
Google GenAI
HTTPX
```

Commands:

```
cd backend
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Or:

```
.\start-server.ps1
```

E. Code Snippets from Various Sources / References

No code snippets were found to be copied directly from external sources. The code was constructed as part of the project using:

- Official Next.js documentation for routing, client components, useRouter, and allowedDevOrigins.
- React documentation for hooks such as useState, useEffect, useMemo, and useCallback.
- FastAPI documentation for routers, dependencies, and CORS middleware.
- PyMongo documentation for MongoDB connection, collections, and indexes.
- Leaflet / React Leaflet documentation for map visualization.
- jsPDF / jspdf-autotable documentation for PDF exports.

General references:

```
https://nextjs.org/docs
https://react.dev/reference/react
https://fastapi.tiangolo.com/
https://pymongo.readthedocs.io/
https://leafletjs.com/
https://react-leaflet.js.org/
https://github.com/parallax/jsPDF
https://github.com/simonbengtsson/jsPDF-AutoTable
```

F. Prompts Used with AI Tools

The project utilized Codex/AI for writing, debugging, and explaining code. Examples of core prompts include:

```
Check if the update driver score function is correct.
Edit the update score function to also update history.
The score should be updated when a driver completes a delivery based on the current
mission only.
Use the history to display a score chart by date and by mission.
Add a total km by driver chart to the dispatcher report.
Add a total km panel to the driver dashboard.
Add an online dispatcher panel and an icon for sending a message.
Add an option to disable notifications and mute sound.
Add dismiss all and a filter to the alerts page.
Add register as driver to the login page and send it for dispatcher approval.
Add a shared calendar for the driver and dispatcher.
Move the driver's calendar to a separate page and add it to the menu.
Fix availability marking and removing with temporary colors.
Remove alerts about offline drivers.
Add brief comments above core frontend functions.
Explain how messages work from the component down to the DB.
```

G. Usernames and Passwords

Users are defined in: backend/app/core/security.py

User Type	Username	Password	Notes
Dispatcher	dispatcher1	dispatch123	Operations Dispatcher
Dispatcher	dispatcher2	dispatch123	Field Dispatcher
Driver	driver_sedan	drive123	driver_id: drv_001, car_type: sedan
Driver	driver_suv	drive123	driver_id: drv_002, car_type: suv
Driver	driver_van	drive123	driver_id: drv_003, car_type: van
Driver	driver_refrigerated	drive123	driver_id: drv_004, car_type: refrigerated_van

Passwords are saved in the DB as hashes using:

```
pwd_context.hash(password)
```

And authentication verification is handled via:

```
pwd_context.verify(password, password_hash)
```

H. Example Core System Flow - Messages

```
MessagesPage
→ getMessageParticipants + getMessageConversations
→ MessageDirectory displays contacts
→ Click on message icon
→ MessageConversationPage
→ getMessages(participantRole, participantId)
→ setMessages(thread)
→ MessageThread displays chat bubbles
→ sendMessage(...)
→ POST /api/messages
→ backend create_message
→ db.create_message
→ MongoDB messages collection
```

I. Submission Notes

- The Frontend satisfies the criteria: React / Next.js with Tailwind.
- The Backend does not use Express; it uses FastAPI instead. Architecturally, it satisfies the remote web backend server requirement: an independent API server connecting to MongoDB.
- If Express is explicitly mandatory, note that there is a technological divergence: FastAPI was chosen instead of Express.